

Principal and Identity Objects

09/15/2021

ⓘ Note

This article applies to Windows.

For information about ASP.NET Core, see [ASP.NET Core Security](#).

Managed code can discover the identity or the role of a principal through a [IPrincipal](#) object, which contains a reference to an [IIdentity](#) object. It might be helpful to compare identity and principal objects to familiar concepts like user and group accounts. In most network environments, user accounts represent people or programs, while group accounts represent certain categories of users and the rights they possess. Similarly, .NET identity objects represent users, while roles represent memberships and security contexts. In .NET, the principal object encapsulates both an identity object and a role. .NET applications grant rights to the principal based on its identity or, more commonly, its role membership.

Identity Objects

The identity object encapsulates information about the user or entity being validated. At their most basic level, identity objects contain a name and an authentication type. The name can either be a user's name or the name of a Windows account, while the authentication type can be either a supported logon protocol, such as Kerberos V5, or a custom value. .NET defines a [GenericIdentity](#) object that can be used for most custom logon scenarios and a more specialized [WindowsIdentity](#) object that can be used when you want your application to rely on Windows authentication. Additionally, you can define your own identity class that encapsulates custom user information.

The [IIdentity](#) interface defines properties for accessing a name and an authentication type, such as Kerberos V5 or NTLM. All **Identity** classes implement the **IIdentity** interface. There is no required relationship between an **Identity** object and the Windows process token under which a thread is currently executing. However, if the **Identity** object is a **WindowsIdentity** object, the identity is assumed to represent a Windows security token.

Principal Objects

The principal object represents the security context under which code is running. Applications that implement role-based security grant rights based on the role associated with a principal object. Similar to identity objects, .NET provides a [GenericPrincipal](#) object and a [WindowsPrincipal](#) object. You can also define your own custom principal classes.

The [IPrincipal](#) interface defines a property for accessing an associated **Identity** object as well as a method for determining whether the user identified by the **Principal** object is a member of a given role. All **Principal** classes implement the **IPrincipal** interface as well as any additional properties and methods that are necessary. For example, the common language runtime provides the **WindowsPrincipal** class, which implements additional functionality for mapping group membership to roles.

A **Principal** object is bound to a call context ([CallContext](#)) object within an application domain ([AppDomain](#)). A default call context is always created with each new **AppDomain**, so there is always a call context available to accept the **Principal** object. When a new thread is created, a **CallContext** object is also created for the thread. The **Principal** object reference is automatically copied from the creating thread to the new thread's **CallContext**. If the runtime cannot determine which **Principal** object belongs to the creator of the thread, it follows the default policy for **Principal** and **Identity** object creation.

A configurable application domain-specific policy defines the rules for deciding what type of **Principal** object to associate with a new application domain. Where security policy permits, the runtime can create **Principal** and **Identity** objects that reflect the operating system token associated with the current thread of execution. By default, the runtime uses **Principal** and **Identity** objects that represent unauthenticated users. The runtime does not create these default **Principal** and **Identity** objects until the code attempts to access them.

Trusted code that creates an application domain can set the application domain policy that controls construction of the default **Principal** and **Identity** objects. This application domain-specific policy applies to all execution threads in that application domain. An unmanaged, trusted host inherently has the ability to set this policy, but managed code that sets this policy must have the [System.Security.Permissions.SecurityPermission](#) for controlling domain policy.

When transmitting a **Principal** object across application domains but within the same process (and therefore on the same computer), the remoting infrastructure copies a reference to the **Principal** object associated with the caller's context to the callee's context.

See also

- [How to: Create a WindowsPrincipal Object](#)
- [How to: Create GenericPrincipal and GenericIdentity Objects](#)
- [Replacing a Principal Object](#)
- [Impersonating and Reverting](#)
- [Role-Based Security](#)
- [Key Security Concepts](#)
- [ASP.NET Core Security](#)